

Category Theory for Dan, Mike, and Ulrik

Groupoid Infinity

May 2026

Abstract

This article describes the design and implementation of the categorical libraries across the **Dan** presentation front-end and the **Mike** (DTT) and **Ulrik** (STT) typechecking environments. We focus on the transition from first-order Directed Type Theory (**Mike**), which utilizes ends, coends, and path-induction primitives, to the minimal primitives of Simplicial Type Theory (**Ulrik**), where all functions are functorial and natural transformations/Yoneda maps are formulated directly using dependent functions and reflexivity paths. We also outline the compilation and verification workflow via the `lift` utility.

Contents

1	Introduction	2
2	Definitional Duality	2
3	Directed Type Theory Library (Mike)	2
4	Simplicial Type Theory Library (Ulrik)	3
5	The Lifting and Verification Workflow	4

1 Introduction

The categorical libraries in our system are located in three corresponding directories:

- `library/dan/`: Combinatorial definitions and simplicial complex shapes.
- `library/mike/`: Precategories, functors, natural transformations, adjunctions, and Yoneda coends formulated in Directed Type Theory (**DTT**).
- `library/ulrik/`: Categories, functors, natural transformations, equivalences, and Yoneda maps formulated in Simplicial Type Theory (**STT**).

2 Definitional Duality

In both **Ulrik** and **Mike** engines, the opposite category operation $(\cdot)^{\text{op}}$ is a definitional involution on categories:

$$(\mathcal{C}^{\text{op}})^{\text{op}} \equiv \mathcal{C} \quad (1)$$

Furthermore, the dual of a natural transformation is definitionally involutive:

$$(\alpha^{\text{op}})^{\text{op}} \equiv \alpha \quad (2)$$

This allows us to obtain dual category-theoretic statements "for free" upon evaluation in the typechecker, eliminating the need for explicit transport along isomorphisms.

3 Directed Type Theory Library (Mike)

The DTT engine (**Mike**) treats categories as first-order parameter structures and represents precategory structures using the $\text{hom}(\mathcal{C}, x, y)$ primitive.

- **Morphism Composition**: Defined via path-induction J :

$$g \circ f \equiv J(w_1.w_2.\text{hom}(\mathcal{C}, x, w_2), z, f, y, z, g) \quad (3)$$

- **Integrals**: DTT supports ends and coends natively. Natural transformations between functors F and G are defined as the end over the homs of the components:

$$\text{Nat}(F, G) \equiv \int_{w:\mathcal{C}} \text{hom}(\mathcal{D}, F(w), G(w)) \quad (4)$$

- **Yoneda Coend:** The Yoneda lemma in DTT is expressed using coends and tensor products:

$$\text{YonedaCoend}(x, z) \equiv \int^{y:\mathcal{C}} (\text{hom}(\mathcal{C}, x, y) \otimes \text{hom}(\mathcal{D}, z, F(y))) \quad (5)$$

The covariant Yoneda map is defined using contravariant path induction:

$$\text{yoneda_map} \equiv \lambda e. \text{let } \langle y, t \rangle := e \text{ in let } f \otimes u := t \text{ in } J_{\text{contra}}(w.F(w, z), u, y, f) \quad (6)$$

4 Simplicial Type Theory Library (Ulrik)

In Simplicial Type Theory (**STT**) as implemented in **Ulrik**, we adopt a deliberately minimalistic set of primitives following the LICS 2025 Gratzer-Weinberger-Buchholtz paper. Primitives such as ends, coends, tensors, and path-induction operators (J_{cov} , J_{contra}) are omitted.

1. **Functors:** All functions between types are automatically functorial in STT. Hence, a functor is simply represented as a function $F : A \rightarrow B$.
2. **Natural Transformations:** A natural transformation between F and G is represented as a dependent function mapping objects to paths in B :

$$\text{Nat}(F, G) \equiv (w : A) \rightarrow \text{hom}(B, F(w), G(w)) \quad (7)$$

The identity natural transformation for a functor F is defined using the constant reflexivity path $\lambda t.F(w)$ for each component:

$$\text{id_nat} \equiv \lambda w. \lambda t. F(w) \quad (8)$$

3. **Presheaves and Yoneda Map:** A presheaf F on category C is a function $C \rightarrow U$, where U is the universe of spaces (types). Because morphisms in U are functions ($\rightarrow$), natural transformations between presheaves F and G have type:

$$\text{PshNat}(F, G) \equiv (w : C) \rightarrow F(w) \rightarrow G(w) \quad (9)$$

The representable functor for an object c is $\text{Yo}(c) \equiv \lambda c'. \text{hom}(C, c, c')$. The Yoneda map has type:

$$\text{yoneda_map} : \text{PshNat}(\text{Yo}(c), F) \rightarrow F(c) \quad (10)$$

This is defined by evaluating the natural transformation at the constant identity path:

$$\text{yoneda_map}(\eta) \equiv \eta(c, \lambda t.c) \quad (11)$$

5 The Lifting and Verification Workflow

The operational layer (**Dan**) allows for a presentation-oriented description of simplicial structures. Using the ‘lift’ binary, these definitions are automatically translated into type-theoretic definitions:

```
$ lift -stt library/dan/simplex.dan
$ lift -dirtt library/dan/simplex.dan
```

Verification is performed independently using the two separate typecheckers:

```
$ dune exec ulrik library/ulrik/book.ulrik
$ dune exec mike library/mike/book.mike
$ dune exec dan library/dan/book.dan
```

References

- [1] Emily Riehl and Michael Shulman. *A type theory for synthetic ∞ -categories*. Higher Structures, 2017.
- [2] Daniel Gratzer, Jonathan Weinberger, and Ulrik Buchholtz. *The Yoneda embedding in simplicial type theory*. LICS 2025.